



Python Modules and Date-Time Module

Section 1: Learn

What is a Module?

A **module** in Python is a file containing Python definitions and statements. It helps you organize code into smaller, manageable parts. You can import and use the code written in a module into other Python programs.

Why Use Modules?

- To avoid repeating code.
- To organize code logically, making it easier to maintain.
- To make code reusable.

Types of Modules

1. **Built-in Modules:** These modules are pre-installed with Python, and you don't have to write them yourself.
2. **User-defined Modules:** You create these modules yourself for your specific program needs.

Interesting Anecdote

Python modules come from the idea of "libraries" in older programming languages like C, but Python made it much easier and more efficient to break down code. By using modules, you can reuse code across different programs without rewriting it each time.



Built-in Modules

Python provides several **built-in modules** which are pre-installed with Python, such as:

- **math**: For mathematical functions like `sqrt()`, `sin()`, etc.
- **random**: To generate random numbers.
- **sys**: For system-specific parameters and functions.

User-defined Modules

A **user-defined module** is a Python file (with `.py` extension) that you create to hold functions, variables, and classes for a specific program. You can import and use these functions in other programs.

How to Create a User-defined Module?

1. Create a Python file, e.g., `mymodule.py`.
2. Write some code inside it (functions, classes, etc.).
3. Import this file in another Python script using the `import` keyword.

What is a Package?

A **package** is a collection of related modules. Packages help organize modules in a hierarchical structure.

Example:

- **numpy** is a package that contains several modules related to numerical operations.



Date and Time Module

The **datetime module** in Python provides classes for manipulating dates and times. It allows you to perform operations like getting the current date and time, calculating time differences, formatting dates, etc.

Section 2: Practice

1. Built-in Modules Example

Importing and Using the **math** module:

```
import math
print(math.sqrt(16)) # Prints the square root of 16
```

Output:

```
4.0
```

Importing and Using the **random** module:

```
import random
print(random.randint(1, 100)) # Generates a random number between 1 and 100
```

Output:

```
43
```

2. User-defined Module Example

Creating a Module:

Create a file named **greetings.py**:



```
def greet(name):  
    print(f"Hello, {name}!")
```

Using the Module:

In another Python file:

```
import greetings  
greetings.greet("Ravi") # Calls the greet function from greetings
```

Output:

```
Hello, Ravi!
```

3. Working with Date and Time Module

Getting the Current Date and Time:

```
import datetime  
now = datetime.datetime.now()  
print(f"Current Date and Time: {now}")
```

Output:

```
Current Date and Time: 2025-05-03 14:22:01.123456
```

Formatting Date and Time:

```
import datetime  
today = datetime.datetime.today()  
formatted_date = today.strftime("%d/%m/%Y")  
print(f"Formatted Date: {formatted_date}")
```



Output:

Formatted Date: 03/05/2025

Section 3: Know More (FAQs)

Q1. What is the difference between a module and a package in Python?

- A **module** is a single file containing Python code (functions, variables, classes).
- A **package** is a collection of modules grouped together in a directory, often used to organize related modules.

Q2. Can we import specific functions from a module?

Yes, you can import specific functions or classes from a module using:

```
from math import sqrt
print(sqrt(25)) # Only imports sqrt function
```

Q3. How can I list all functions in a module?

You can use the `dir()` function to list all functions and variables in a module:

```
import math
print(dir(math))
```

Q4. Can a user-defined module be used from a different directory?

Yes, you can. Use the `sys.path` to add the directory containing your module to the search path:

```
import sys
sys.path.append('path_to_your_module')
```



```
import mymodule
```

Q5. What is the use of the **datetime** module?

The **datetime** module allows you to manipulate dates and times, calculate time differences, and format dates for display.

*Modules, packages, and the **datetime** module provide the foundation for writing clean, efficient, and maintainable Python code. By using built-in and custom modules, you can organize your projects better and save time by reusing code.*